Artificial Intelligence
Problem Solving

Chapter-3

Problem Solving:

Problem solving in AI involves the application of computational algorithms and techniques to address complex problems or tasks. AI systems are designed to emulate human-like problem-solving capabilities, enabling them to analyze, reason, and generate solutions in various domains. Here is a general framework for problem solving in AI:

- **Problem Definition**: Clearly define the problem that needs to be solved. This involves understanding the problem domain, identifying the desired outcome, and specifying any constraints or limitations.
- **Data Gathering**: Collect relevant data that is necessary to solve the problem. This can include structured data (e.g., databases, spreadsheets) or unstructured data (e.g., text, images, videos). The quality and quantity of the data play a crucial role in the effectiveness of the solution.
- **Preprocessing and Feature Extraction**: Clean the data and transform it into a suitable format for analysis. This step may involve removing noise, handling missing values, normalizing data, or extracting meaningful features from raw data.
- **Algorithm Selection**: Choose an appropriate algorithm or technique that is suitable for the problem at hand. The selection can depend on factors such as the nature of the problem (classification, regression, optimization), available data, computational resources, and desired performance.

Model Training: Train the selected algorithm using the prepared data. This typically involves feeding the algorithm with labeled examples (in supervised learning) or allowing it to explore and learn from the data (in unsupervised or reinforcement learning).
Model Evaluation: Assess the performance of the trained model using suitable evaluation metrics. This step helps determine the model's accuracy, precision, recall, or other relevant measures to measure its effectiveness in solving the problem.
Fine-tuning and Iteration: If the model's performance is not satisfactory, fine-tune the parameters or adjust the algorithm. This step may involve retraining the model with additional data, tweaking hyper parameters, or changing the algorithm altogether.
Deployment: Once the model meets the desired performance level, deploy it to the production environment. This can involve integrating the AI system into existing software or hardware infrastructure, making it accessible to end-users or other systems.
Monitoring and Maintenance: Continuously monitor the deployed AI system to ensure it performs as expected. This involves tracking its performance, identifying potential issues or biases, and periodically updating the model or retraining it as new data becomes available.
Continuous Improvement: Seek opportunities to enhance the AI system's performance over time. This can involve collecting user feedback, incorporating new techniques or algorithms, or adapting the system to evolving user needs and changing problem requirements.

Defining problems as a state space search

- A **state space** is a way to mathematically represent a problem by defining all the possible states in which the problem can be. This is used in search algorithms to represent the initial state, goal state, and current state of the problem. Each state in the state space is represented using a set of variables.
- The **efficiency** of the search algorithm greatly depends on the size of the state space, and it is important to choose an appropriate representation and search strategy to search the state space efficiently.
- One of the most well-known **state space search algorithms** is the A algorithm. Other commonly used state space search algorithms include **breadth-first search (BFS)**, **depth-first search (DFS)**, **hill climbing**, **simulated annealing**, and **genetic algorithms**.

Features of State Space Search

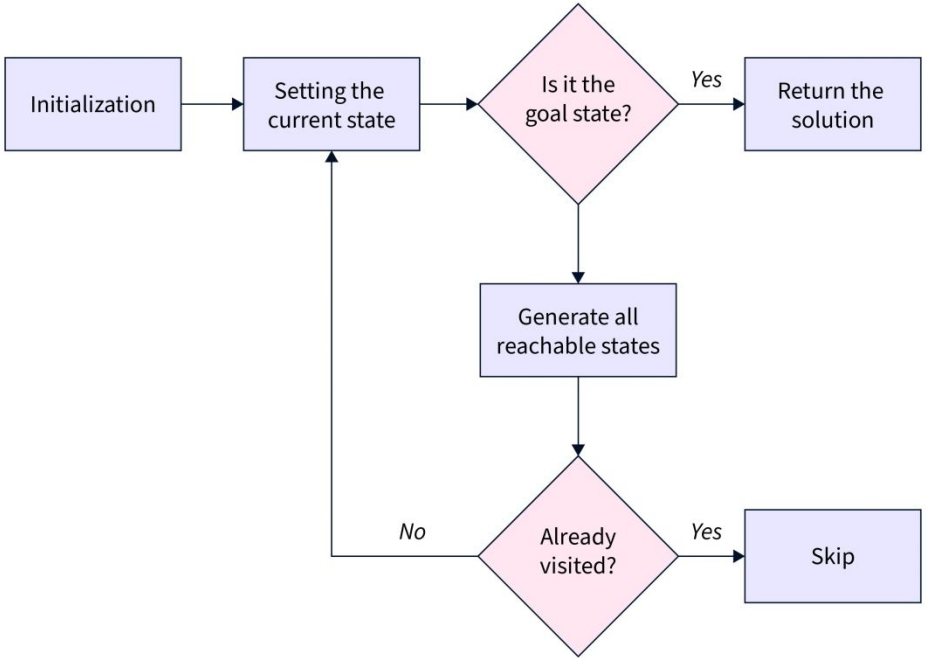
State space search has several features that make it an effective problem-solving technique in Artificial Intelligence. These features include:

- **Exhaustiveness:**
  State space search explores all possible states of a problem to find a solution.
- **Completeness:**
  If a solution exists, state space search will find it.
- **Optimality: (**satisfactory**)**
  Searching through a state space results in an optimal solution.
- **Uninformed and Informed Search:**
  State space search in artificial intelligence can be classified as uninformed if it provides additional information about the problem.

**In contrast, informed search uses additional information, such as heuristics, to guide the search process.

Steps in State Space Search

The steps involved in state space search are as follows:



SCALER Topics

To begin the search process, we set the current state to the initial state.
We then check if the current state is the goal state. If it is, we terminate the algorithm and return the result.
If the current state is not the goal state, we generate the set of possible successor states that can be reached from the current state.
For each successor state, we check if it has already been visited. If it has, we skip it, else we add it to the queue of states to be visited.
Next, we set the next state in the queue as the current state and check if it's the goal state. If it is, we return the result. If not, we repeat the previous step until we find the goal state or explore all the states.
If all possible states have been explored and the goal state still needs to be found, we return with no solution.

State Space Representation

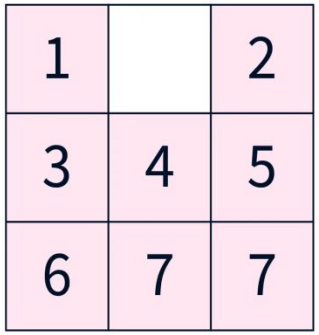
State space Representation involves defining an INITIAL STATE and a GOAL STATE and then determining a sequence of actions, called states, to follow.

- **State:**
  A state can be an Initial State, a Goal State, or any other possible state that can be generated by applying rules between them.
- **Space:**
  In an AI problem, space refers to the exhaustive collection of all conceivable states.
- **Search:**
  This technique moves from the beginning state to the desired state by applying good rules while traversing the space of all possible states.
- **Search Tree:**
  To visualize the search issue, a search tree is used, which is a tree-like structure that represents the problem. The initial state is represented by the root node of the search tree, which is the starting point of the tree.
- **Transition Model:**
  This describes what each action does, while Path Cost assigns a cost value to each path, an activity sequence that connects the beginning node to the end node. The optimal option has the lowest cost among all alternatives.

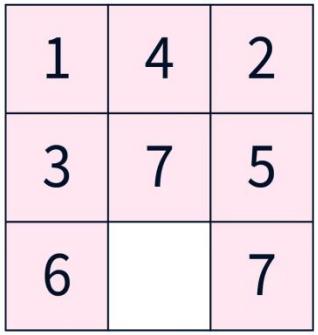
Example of State Space Search

- The **8-puzzle** problem is a commonly used example of a state space search. It is a sliding puzzle game consisting of 8 numbered tiles arranged in a 3x3 grid and one blank space. The game aims to rearrange the tiles from their initial state to a final goal state by sliding them into the blank space.
- To represent the state space in this problem, we use the nine tiles in the puzzle and their respective positions in the grid. Each state in the state space is represented by a 3x3 array with values ranging from 1 to 8, and the blank space is represented as an empty tile.
- The initial state of the puzzle represents the starting configuration of the tiles, while the goal state represents the desired configuration. **Search algorithms** utilize the state space to find a sequence of moves that will transform the initial state into the goal state.

- The **8-puzzle** problem is a commonly used example of a state space search. It is a sliding puzzle game consisting of 8 numbered tiles arranged in a 3x3 grid and one blank space. The game aims to rearrange the tiles from their initial state to a final goal state by sliding them into the blank space.
- To represent the state space in this problem, we use the nine tiles in the puzzle and their respective positions in the grid. Each state in the state space is represented by a 3x3 array with values ranging from 1 to 8, and the blank space is represented as an empty tile.
- The initial state of the puzzle represents the starting configuration of the tiles, while the goal state represents the desired configuration. **Search algorithms** utilize the state space to find a sequence of moves that will transform the initial state into the goal state.



Current State

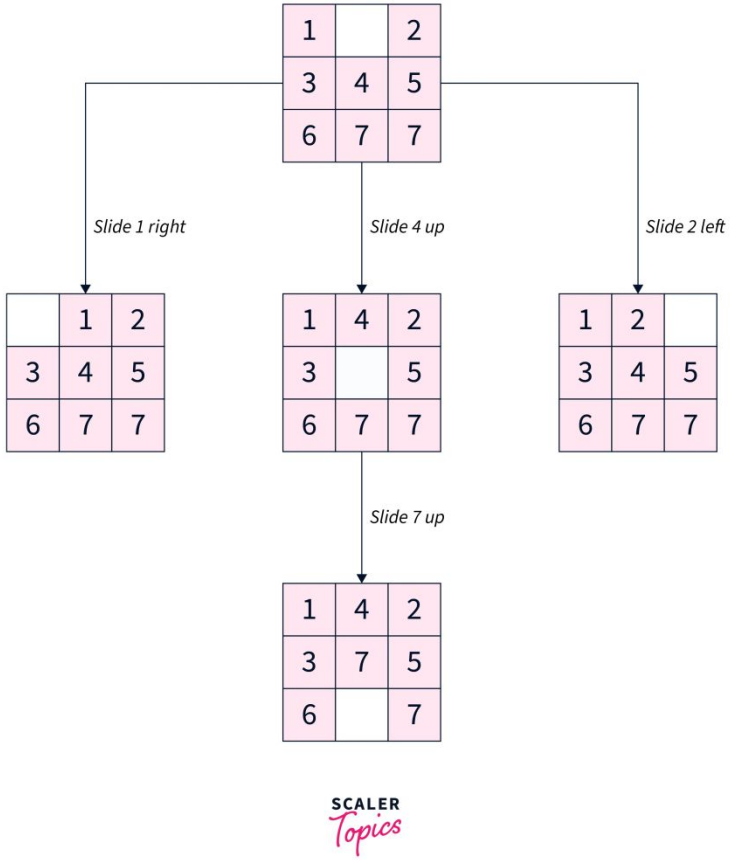


Target State

SCALER Topics

This algorithm guarantees a solution but can become very slow for larger state spaces. Alternatively, other algorithms, such as **A search**, use heuristics to guide the search more efficiently.

Our objective is to move from the current state to the target state by sliding the numbered tiles through the blank space. Let's look closer at reaching the target state from the current state.



To summarize, our approach involved exhaustively exploring all reachable states from the current state and checking if any of these states matched the target state.

Applications of State Space Search

- State space search algorithms are used in various fields, such as robotics, game playing, computer networks, operations research, bioinformatics, cryptography, and supply chain management. In artificial intelligence, state space search algorithms can solve problems like **path finding**, **planning**, and **scheduling**.
- They are also useful in planning robot motion and finding the best sequence of actions to achieve a goal. In games, state space search algorithms can help determine the best move for a player given a particular game state.
- **State space search algorithms** can optimize routing and resource allocation in computer networks and operations research.
- In **Cryptography**, state space search algorithms are used to break codes and find cryptographic keys.

Problem Formulation in Artificial Intelligence (AI)

- Every problem should be properly formulated in artificial intelligence. Problem formulation is very important before applying any search algorithm. Every algorithm demands problem is specific form. Before problem formulation it is very important to know components of problem.
- Problem Formulation is the process of deciding what actions and states to consider, given goal. Simply a formulation of the problem is getting in term of the

  Initial State

  Actions

  Transition Model

  Goal Test

  Path Cost

General Problem Solving Components

In AI one must identify components of problems, which are:-

- Problem Statement
  - Definition
  - Limitation or Constraints or Restrictions
- Problem Solution
- Solution Space
- Operators

Definition of Problem

The information about what is to be done? Why it is important to build AI system? What will be the advantages of proposed system? For example "***I want to predict the price of house using AI system***".

Problem Limitation

There always some limitations while solving problems. All these limitations or constraints must be fulfill while creating system. For example "***I have only few features, some records are missing. System must be 90% accurate otherwise will be useless***".

Goal or Solution

What is expected from system? The Goal state or final state or the solution of problem is defined here. This will help us to proposed appropriate solution for problem. For example "***we can use some machine learning technique to solve this problem*".**

Solution Space

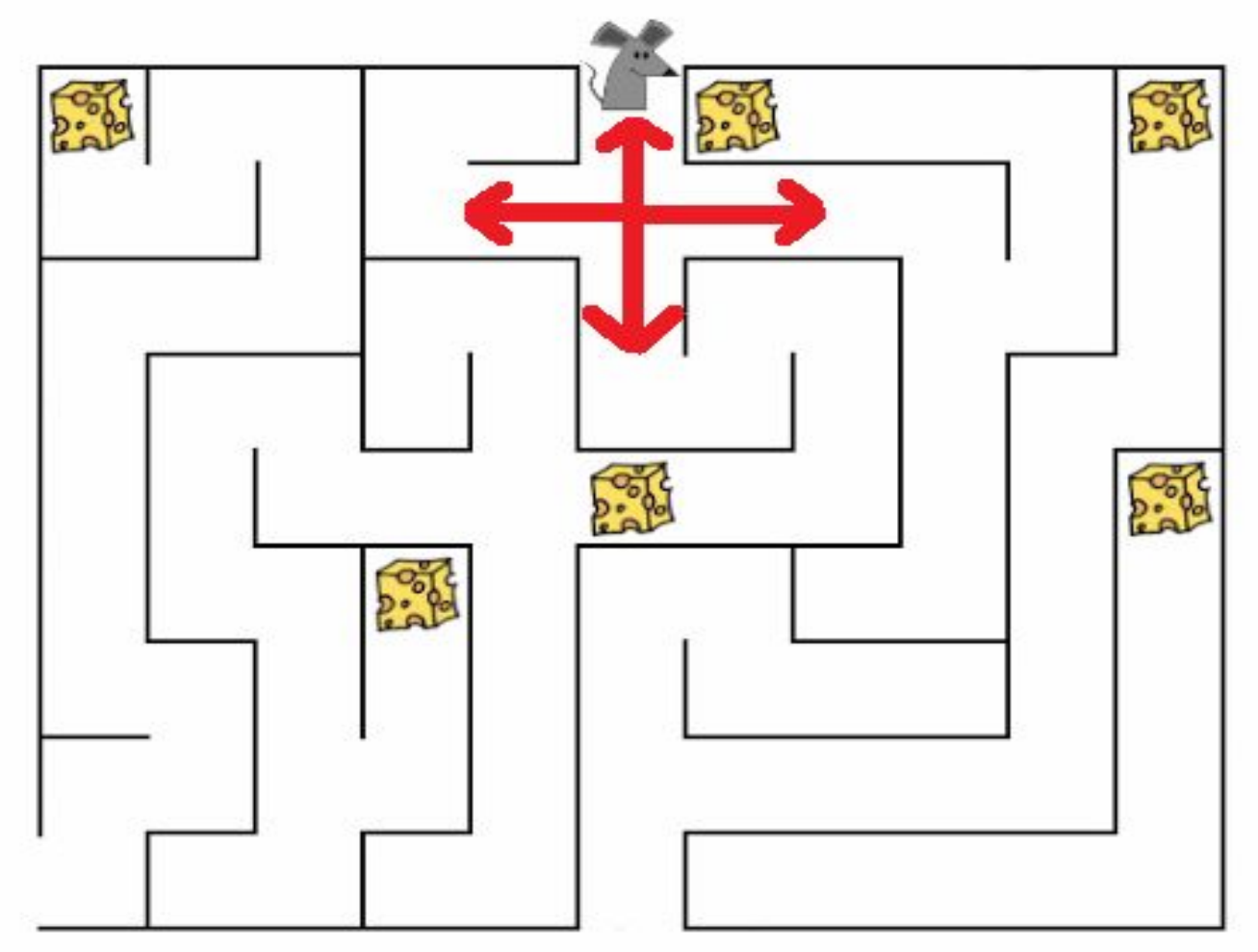
Problem can be solve in many ways. Some solution will be efficient than others. Some will consume less resources, some will be simple etc. There are always alternatives exists. Many possible ways with which we can solve problem is known as Solution Space. For example "***price of house can be predict using many machine learning algorithms***".

Operators

Operators are the actions taken during solving problem. Complete problem is solved using tiny steps or actions and all these consecutive actions leads to solution of problem.

Examples of Problem Formulation

Mouse Path Problem



- **Problem Statement**
  - **Problem Definition:** Mouse is hungry, mouse is in a puzzle where there are some cheese. Mouse will only be satisfied if mouse eat cheese
  - **Problem Limitation:** Some paths are close i-e dead end, mouse can only travel through open paths
- **Problem Solution**: Reach location where is cheese and eat minimum one cheese. There are possible solutions (cheese pieces)
- **Solution Space:** To reach cheese there are multiple paths possible
- **Operators**: Mouse can move in four possible directions, these directions are operators or actions which are UP, DOWN, LEFT and RIGHT

Water Jug Problem

Problem Statement

Problem Definition: You have to measure 4 liter (L) water by using three buckets 8L, 5L and 3L.

Problem Limitation: You can only use these (8L, 5L and 3L) buckets

Problem Solution: Measure exactly 4L water

Solution Space: There are multiple ways doing this.

Operators: Possible actions are fill water in any bucket and remove water from any bucket.

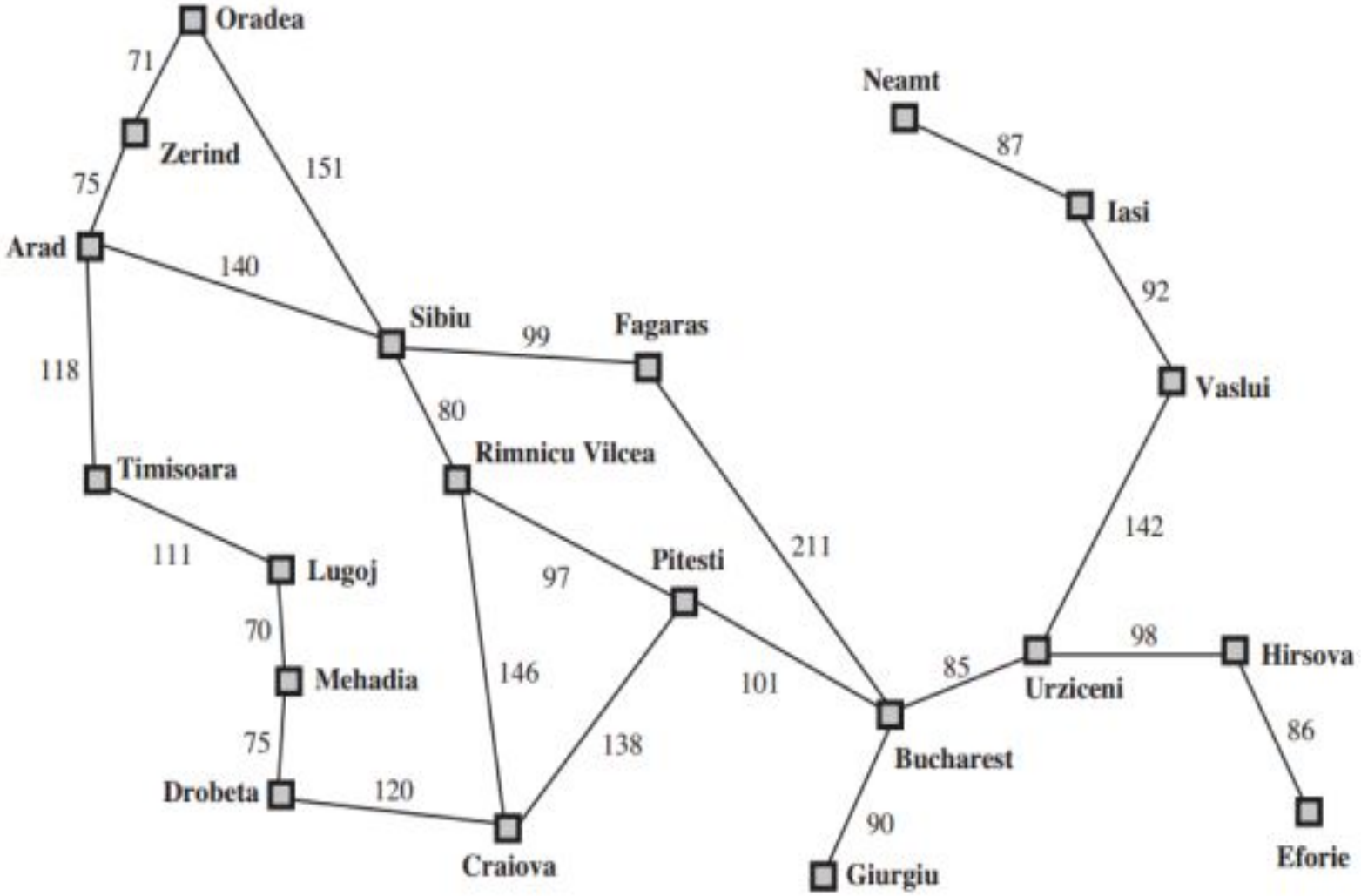
Measure 4L Using 3 Buckets

Path Finding Problem

- **Problem Statement**

    - **Definition:** Going from Arad to Bucharest in given map
    - **Limitation:** Must travel from location to other if there is path

- **Problem Solution**: Reach Bucharest
- **Solution Space:** There are multiple paths to reach Bucharest.
- **Operators**: Move to other locations


Oradea
71
Neamt
Zerind
87
151
75
Iasi
Arad
140
92
Sibiu
99
Fagaras
118
Vaslui
80
Rimnicu Vilcea
Timisoara
142
111
Pitesti
211
Lugoj
97
98
Hirsova
70
Mehadia
146
101
85
Urziceni
86
75
138
Bucharest
Drobeta
120
90
Craiova
Eforie
Giurgiu

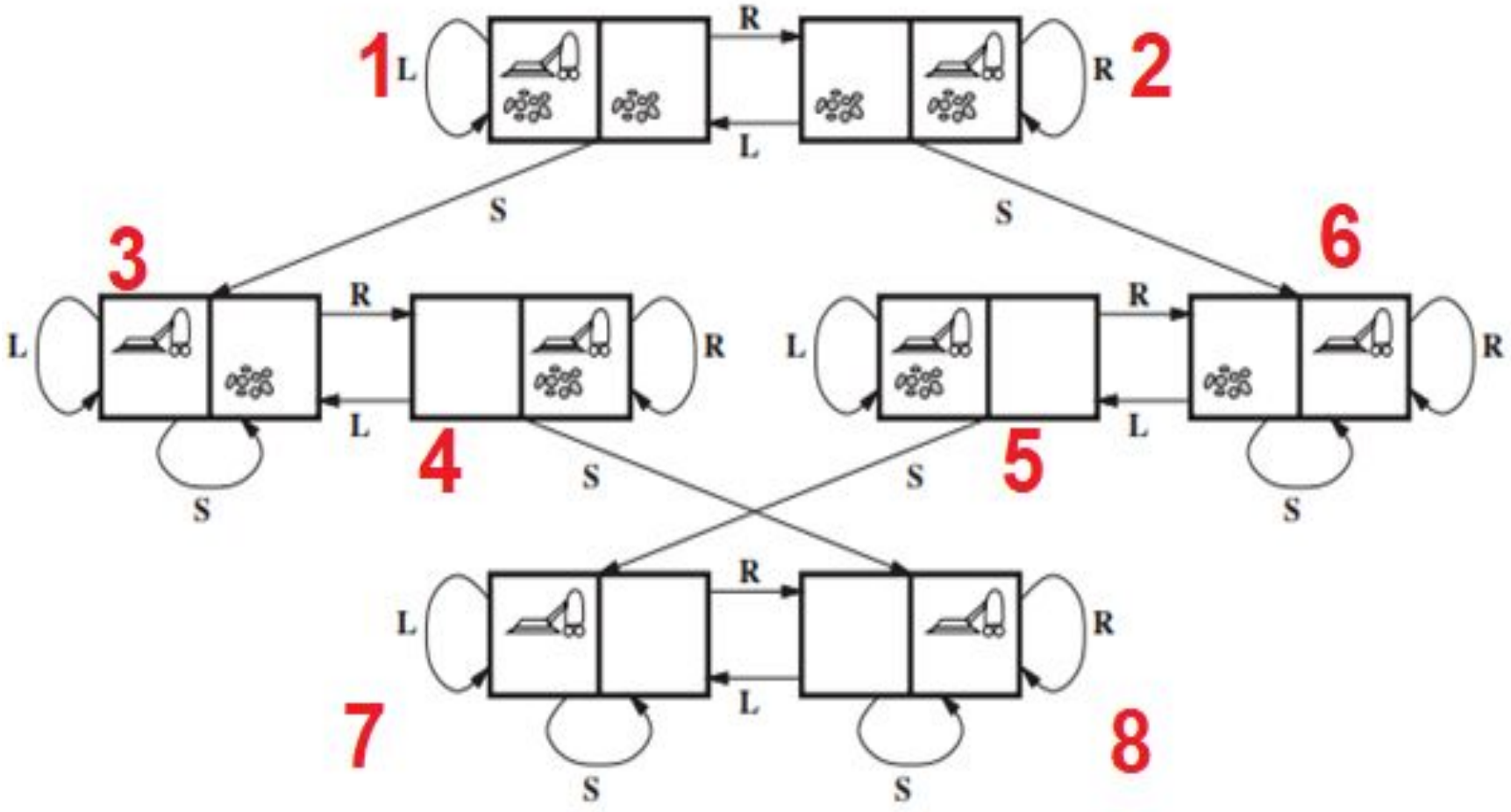
Well-defined Problems and Solution

Problem solving components discussed above are applicable to any problem. For AI system implementation, problem must be well defined. A well-defined problem must have five components:-

- **Initial State**: Start point of problem
- **Final State**: The finish point of problem. Aka Goal or solution state
- **States**: Total states in problem
- **Transition Model**: How one can shift from one state to another
- **Actions**: Actions set, used to move from one state to another
- **Path Cost**: What is total effort (cost) from initial state to final state

Examples of Well-Defined Problems

Vacuum World



- **States**: Agent can be location A or B, hence 2 possibilities. Dirt can be in none, one or both location, and hence 4 possibilities. Thus, there are total $2 \times 2^2 = 8$ possible world states. For environment with n locations has $(n * 2^n)$ states.

Total possible states=Agent's location states×Dirt presence states

Total possible states=2×4=8Total possible states=2×4=8

Agent in A, No dirt

Agent in A, Dirt in A

Agent in A, Dirt in B

Agent in A, Dirt in A and B

Agent in B, No dirt

Agent in B, Dirt in A

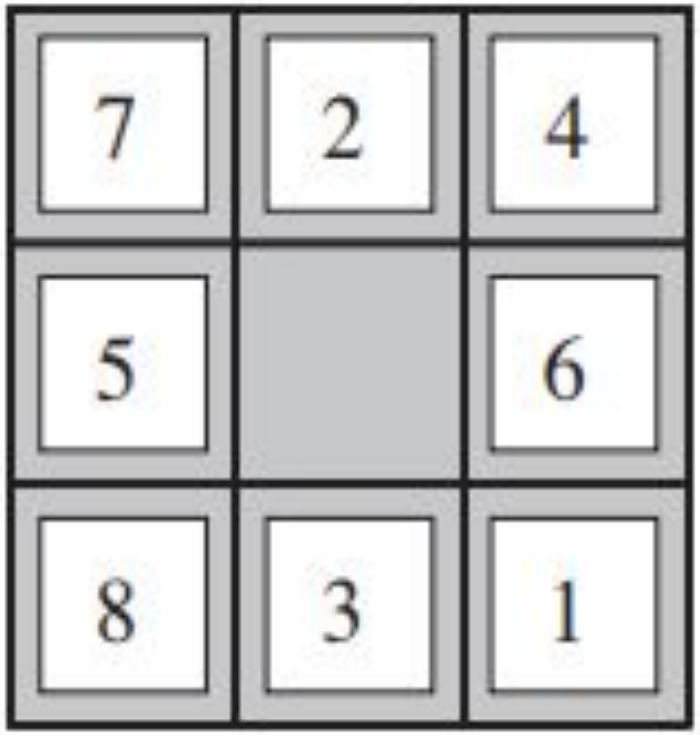
Agent in B, Dirt in B

Agent in B, Dirt in A and B

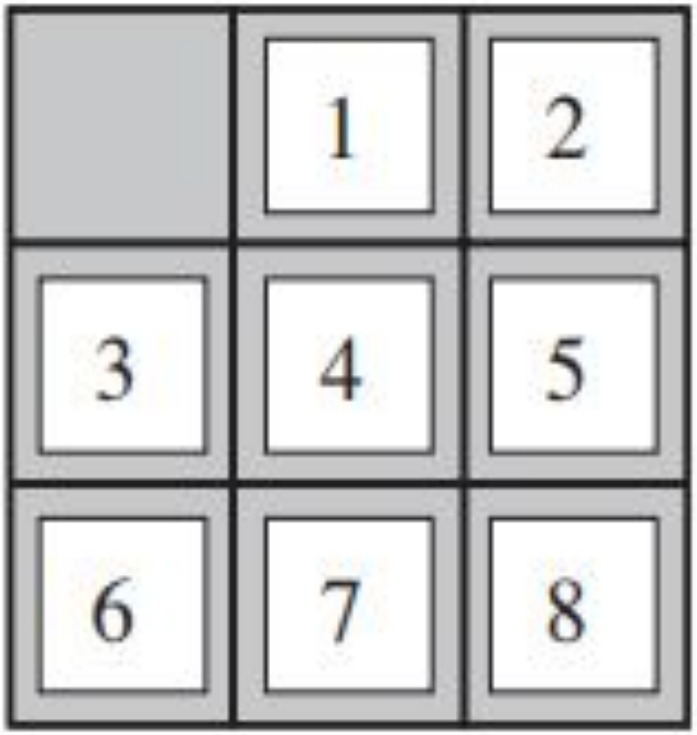
- **Initial state**: Any state can be designated as the initial state.
- **Actions**
  - Move Left
  - Move Right
  - And Suck or Clean
- **Transition model**: This is a function which define transition from one state to another. Consider state 1 in following diagram is initial state. There 3 actions possible
  - If it move LEFT (L) then it will remain in same state
  - and If it move RIGHT (R), agent will be in state 2
  - If it suck or clean, it will be in state 3
- In this way complete graph is created. Transition function be defined as

Examples of Well-Defined Problems

8 Puzzle or Slide Puzzle



Start State

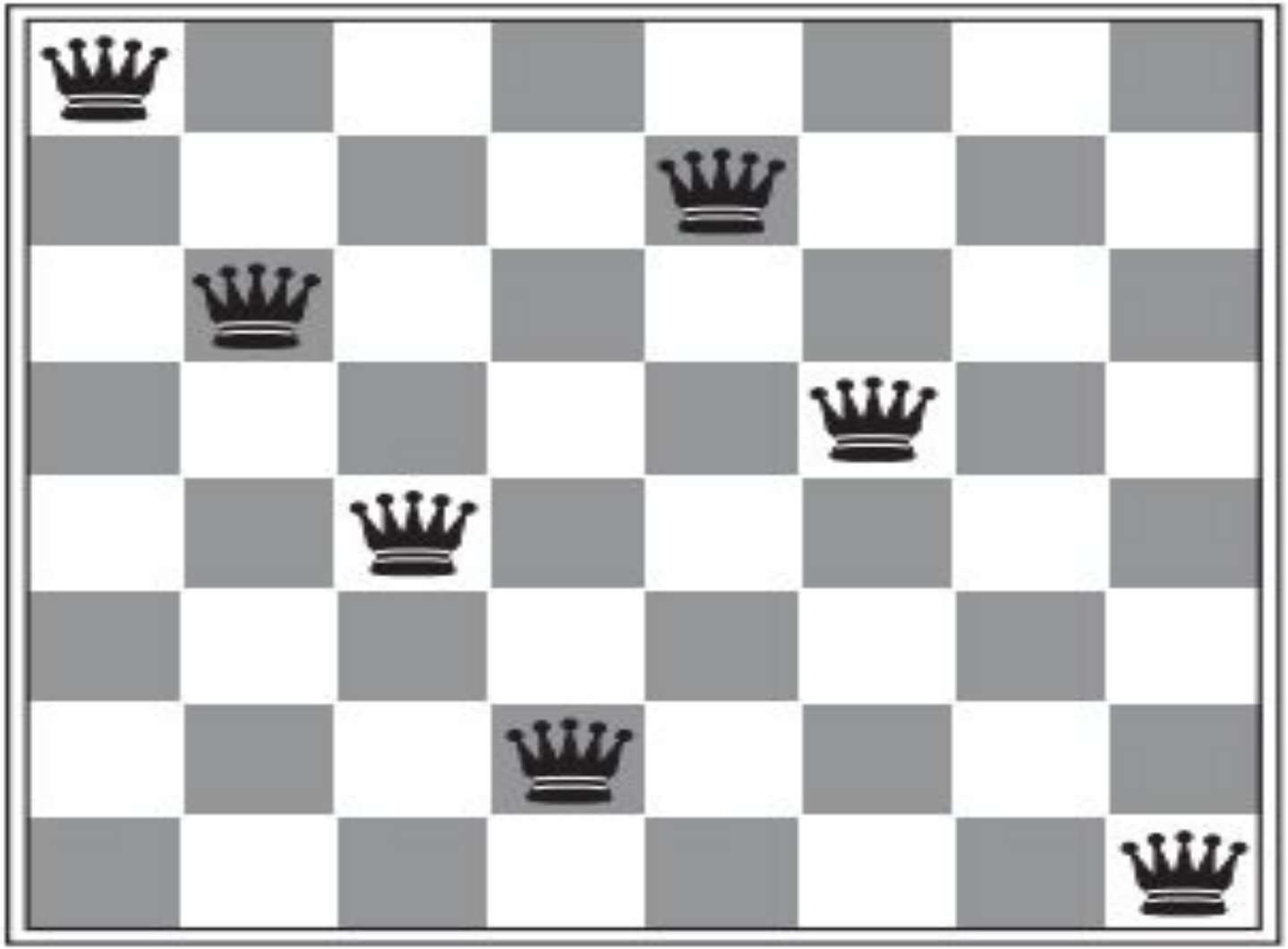


Goal State

- **States**: A state description specifies the location of each of the eight tiles and the blank in one of the nine squares.
- **Initial state**: Any random shuffled state can be designated as initial state
- **Actions**:
  - Slide Left
  - or Slide Right
  - or Slide Up
  - And Slide Down
- **Transition model**: Given a state and action, this returns the resulting state
- **Goal test**: This checks whether the state matches the goal
- **Path cost**: Each step costs 1

Examples of Well-Defined Problems

8 Queens Problem



- **States**: Any arrangement of 0 to 8 queens on the chess board is a state.
- **Initial state**: No queens on the board or randomly shuffled 8 queens on board
- **Actions**: Add a queen to any empty square or move queens one by one
- **Transition model**: Returns the board with a queen added to the specified square.
- **Goal test**: 8 queens are on the board, none attacked.